

CS253 Assignment 1

Memory Efficient Versioned File Indexer

Venkata Prasun Kallu
Roll Number: 240508

1 Overview

This project implements a **memory efficient versioned file indexer** in C++. The program processes large text files incrementally using a **fixed-size buffer** instead of loading the entire file into memory. Words are extracted from the file as contiguous sequences of alphanumeric characters and are treated in a **case-insensitive** manner.

The system builds a **word-level index** that maps each word to the number of times it appears in the file. Each file corresponds to a **version**, allowing multiple versions to be indexed simultaneously.

The system supports three main queries:

- **Word Query** – returns the frequency of a specific word in a version.
- **Difference Query** – computes the difference in word frequency between two versions.
- **Top-K Query** – returns the K most frequent words in a version.

The design ensures that the program's memory usage depends only on the number of unique words and not on the file size.

2 Command Line Parsing

The program accepts input parameters through the command line. These parameters specify the input files, version names, buffer size, query type, and additional query parameters.

Important arguments include:

- `--file` : input file for single version queries
- `--file1, --file2` : input files for difference query
- `--version, --version1, --version2` : version identifiers
- `--buffer` : buffer size in KB (256–1024)
- `--query` : query type (word, diff, top)
- `--word` : word for word and diff queries
- `--top` : number of results for top-k query

The program validates all arguments and throws exceptions for invalid inputs such as incorrect buffer size or missing parameters.

3 Classes and Their Responsibilities

3.1 Buffered_File_Reader

Responsible for reading the file incrementally using a fixed-size buffer.

Functions

- `Buffered_File_Reader(path, bufferSizeKB)` – constructor that initializes file reading with the given buffer size.
- `nextChunk(string &chunk)` – reads the next chunk of data from the file.
- `bufferSizeKB()` – returns the current buffer size in kilobytes.

This class ensures that the entire file is never loaded into memory.

3.2 Tokenizer

Responsible for converting raw text into individual words.

Functions

- `tokenize(text)` – extracts words from a string and returns them as a vector.
- `isWordChar(c)` – checks whether a character is alphanumeric.

This class also converts words to lowercase to maintain case-insensitive indexing.

3.3 Indexer

Maintains the word-frequency index for each version.

The index is stored using:

```
unordered_map<string, unordered_map<string, long long>>
```

Functions

- `build(version, filePath, bufferKB)` – builds the word index for a file version.
- `getFrequency(version, word)` – returns frequency of a word in a version.
- `getMap(version)` – returns the entire word-frequency map of a version.
- `hasVersion(version)` – checks whether a version exists.

3.4 Query (Abstract Base Class)

Provides a common interface for executing queries.

Functions

- `execute(Indexer &idx)` – performs the query operation.
- `printResult()` – prints the query result.

3.5 WordQuery

Handles queries that return the frequency of a single word.

Functions

- `execute()` – retrieves the frequency of the given word.
- `printResult()` – prints the result.

3.6 DiffQuery

Computes the difference in word frequency between two versions.

Functions

- `execute()` – computes frequency difference.
- `printResult()` – displays the difference.

3.7 TopKQuery

Returns the most frequent words in a version.

Functions

- `execute()` – retrieves and sorts the word-frequency pairs.
- `printResult()` – prints the top K words.

3.8 QueryProcessor

Responsible for executing queries using runtime polymorphism.

Functions

- `run(Query &query)` – executes the query and prints the result.

4 Program Workflow

The execution of the program follows these steps:

1. Command-line arguments are parsed and validated.
2. The program initializes the `Indexer`.
3. The `Buffered_File_Reader` reads the input file chunk by chunk.
4. Each chunk is passed to the `Tokenizer` to extract words.
5. Extracted words are stored in the index using the `Indexer`.
6. A query object (`WordQuery`, `DiffQuery`, or `TopKQuery`) is created.
7. The `QueryProcessor` executes the query.
8. The result, buffer size, and execution time are printed.

5 C++ Features Used

The implementation demonstrates several important C++ programming concepts.

5.1 Inheritance

Inheritance is used through the abstract base class `Query`. Specific query types inherit from this base class and implement their own behavior.

- `WordQuery`
- `DiffQuery`
- `TopKQuery`

Each of these classes extends the base `Query` interface and implements query-specific logic.

5.2 Runtime Polymorphism

Runtime polymorphism is achieved using virtual functions defined in the `Query` class.

```
virtual void execute(Indexer &idx) = 0;
```

Different query types override this function. The `QueryProcessor` executes queries through a reference to the base class, allowing dynamic dispatch.

5.3 Templates

A function template is used for sorting word-frequency pairs.

```
template <typename T> sortDescending()
```

This allows the same sorting function to work with different data types.

5.4 Standard Template Library (STL)

Several STL containers are used to efficiently manage data.

- `vector` – stores words and intermediate results
- `unordered_map` – stores word-frequency mappings
- `pair` – stores word-frequency pairs for sorting

5.5 File Handling

The program uses `ifstream` for reading input files in binary mode.

```
ifstream file;
```

This allows efficient buffered reading of large files.

5.6 Performance Measurement

The execution time of the program is measured using the `chrono` library.

```
chrono::high_resolution_clock
```

This helps evaluate the performance impact of different buffer sizes.

6 Error Handling

The program uses C++ exception handling to manage runtime errors and invalid inputs.

The following situations are handled:

- Invalid buffer size outside the allowed range (256–1024 KB)
- Failure to open input files
- Missing command line arguments
- Invalid query types
- Requests for non-existent versions

Exceptions are thrown using `throw` statements and caught in the `main()` function using a `try-catch` block.

Example:

```
throw invalid_argument("Version not found");
```

This ensures that the program terminates gracefully and provides meaningful error messages to the user.

7 Conclusion

The implemented system efficiently processes large text files while maintaining strict memory limits using buffered reading. The object-oriented design separates responsibilities across multiple classes and enables flexible query execution using polymorphism. The system successfully supports analytical queries while ensuring scalable and memory-efficient indexing.